CrewAI 实战解析：程序2考试系统与混合架构

- 来源: https://class.imooc.com/lesson/5293#mid=105160
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: CrewAI 实战
- 课节: 图文 1-4 CrewAI 实战解析：程序2考试系统与混合架构

1-4 CrewAI 实战解析：程序2考试系统与混合架构

CrewAI 实战解析：程序2考试系统与混合架构

> [!TIP] 阅读指南 上一节我们学习了程序1的Sequential模式，看到了Crew、Agent、Task如何协同工作。本节我们将学习程序2（main.py），它展示了一个重要的架构思想：不是所有业务都需要Crew，传统代码与AI决策可以完美混合。

程序2业务流程回顾

程序2实现了完整的考试流程，包含4个阶段：

```
──────────────────────────────────────────────────

  阶段1：组卷
  • 传统业务：从题库随机抽题
  • 逻辑：min(题库总数，配置数量)
──────────────────────────────────────────────────

  阶段2：学生答题
  • 传统业务：收集学生输入（A/B/C/D）
──────────────────────────────────────────────────

  阶段3：自动批改
  • 传统业务：对比答案，计算分数
  • 无需AI：选择题答案确定
──────────────────────────────────────────────────

  阶段4：AI深度分析（可选，用户询问）
  • AI决策：Manager协调2个Agent
    - Agent1：答案解析专家
    - Agent2：学习诊断教练
──────────────────────────────────────────────────
```

一个常见的错误直觉

看到这4个阶段，很多人会自然地想：“这和程序1一样，应该创建4个Task，然后用Sequential模式顺序执行吧？”

假想的实现（如果全部用Crew）：

假想的实现

```
# ✗ 错误的想法：把所有阶段都交给Crew
class ExamSystem:
    def run_exam(self):
        # 创建4个Task
        task1 = create_paper_task(...)      # 组卷
        task2 = create_answer_task(...)     # 收集答案
        task3 = create_grading_task(...)    # 批改
        task4 = create_analysis_task(...)   # 分析

        # 顺序执行
        crew = Crew(
            agents=[paper_agent, answer_agent, grading_agent, analysis_agent],
            tasks=[task1, task2, task3, task4],
            process=Process.sequential
        )

        result = crew.kickoff()
```

这样做的问题：

这样做的问题

- 浪费Token：组卷、收集答案、对比答案都是确定性逻辑，不需要LLM

浪费Token

- 性能低下：每个Task都要调用LLM，即使是简单的数据库查询

性能低下

- 调试困难：传统代码的逻辑错误和AI的输出问题混在一起

调试困难

- 成本高昂：3个阶段的Token消耗完全是浪费

成本高昂

实际的混合架构

程序2的实际实现采用了混合架构：传统代码负责确定性任务，Crew只在需要AI决策时启用。

整体架构

```
class ExamSystem:
    """考试系统主程序"""

    def __init__(self):
        self.config = load_config()
        self.db = QuestionDatabase()
        self.console = Console()

        # 只在需要AI时才初始化Crew
        self.llm = create_llm()
        self.crew = ExerciseCrew(self.llm)

    def run(self):
        """完整考试流程"""
        # 阶段1-3：传统代码（不用Crew）
        self.create_exam_paper()      # 传统：随机抽题
        self.collect_answers()        # 传统：终端输入
        grading_result = self.grade_answers()  # 传统：对比答案

        # 显示分数
        self._display_grading_result(grading_result)

        # 询问是否需要AI分析
        need_analysis = Confirm.ask("是否需要AI进行答案解析和学习建议？")

        if need_analysis:
            # 阶段4：启用Crew（Hierarchical模式）
            explanation, advice = self.crew.run_analysis(grading_result)
            self.console.print(f"答案解析：\n{explanation}")
            self.console.print(f"学习建议：\n{advice}")
```

这三个阶段完全不需要AI，使用确定性算法即可：

```
class ExamSystem:
    def create_exam_paper(self) -> bool:
        """阶段1：组卷 - 核心算法：random.sample()"""
        exam_count = self.config.exam_count
        total_count = self.db.get_question_count()

        if total_count < exam_count:
            return False

        # 随机抽题
        self.exam_questions = self.db.random_select_questions(exam_count)
        return True

    def collect_answers(self):
        """阶段2：收集答案 - 核心算法：输入验证"""
        self.student_answers = []

        for i, question in enumerate(self.exam_questions, 1):
            self._display_question(i, question)

            # 验证输入
            while True:
                answer = Prompt.ask("请输入答案（A/B/C/D）").upper()
                if answer in ['A', 'B', 'C', 'D']:
                    self.student_answers.append(answer)
                    break

    def grade_answers(self) -> dict:
        """阶段3：批改 - 核心算法：字符串比较"""
        total_score = 0
        details = []

        # 逐题对比
        for question, student_answer in zip(self.exam_questions, self.student_answers):
            is_correct = (student_answer == question.answer)
            score = 10 if is_correct else 0
            total_score += score

            details.append({
                'is_correct': is_correct,
                'score': score
            })
```

```
        return {'total_score': total_score, 'details': details}
```

这三个阶段都是确定性算法（随机抽取、输入验证、字符串比较），用LLM不仅浪费Token，还可能出错。

只有在阶段4（AI深度分析）时，才启用Crew：

```
# crews/exercise_crew.py

class ExerciseCrew:
    def run_analysis(self, grading_result: dict):
        """
        执行AI分析（Hierarchical模式）
        只在这里调用LLM!
        """
        # 创建2个Agent
        explanation_agent = create_explanation_agent(self.llm)
        advisor_agent = create_learning_advisor(self.llm)

        # 创建2个Task（可以并行）
        explanation_task = create_explanation_task(explanation_agent, grading_result)
        advice_task = create_learning_advice_task(advisor_agent, grading_result)

        # Hierarchical模式：Manager协调2个Agent并行工作
        crew = Crew(
            agents=[explanation_agent, advisor_agent],
            tasks=[explanation_task, advice_task],
            process=Process.hierarchical,  # 并行执行
            manager_llm=self.llm,          # Manager使用的LLM
            verbose=True
        )

        result = crew.kickoff()

        # 返回两个Agent的输出
        return result.explanation, result.advice
```

为什么要用Hierarchical?

- 并行执行：答案解析和学习建议可以同时进行，提升速度
- Manager协调：需要一个"项目经理"分配任务给2个Agent

- 创造性任务：需要LLM理解错题原因、生成个性化建议

创造性任务

Hierarchical模式：Manager协调多Agent

在程序2的阶段4，我们使用Hierarchical模式协调2个Agent并行工作。

代码实现

```
# crews/exercise_crew.py

class ExerciseCrew:
    def run_analysis(self, grading_result: dict):
        """执行AI分析（Hierarchical模式）"""
        # 创建2个专家Agent
        explanation_agent = create_explanation_agent(self.llm)
        advisor_agent = create_learning_advisor(self.llm)

        # 创建2个Task
        explanation_task = create_explanation_task(explanation_agent, grading_result)
        advice_task = create_learning_advice_task(advisor_agent, grading_result)

        # Hierarchical模式：Manager协调
        crew = Crew(
            agents=[explanation_agent, advisor_agent],
            tasks=[explanation_task, advice_task],
            process=Process.hierarchical,  # 关键：启用Hierarchical
            manager_llm=self.llm,          # Manager使用的LLM
            verbose=True
        )

        result = crew.kickoff()
        return result.explanation, result.advice
```

执行流程

关键特性

- Manager自动创建：你不需要定义Manager Agent，CrewAI自动生成
- 并行执行：2个Agent同时调用LLM，提升速度

- 结果汇总：Manager负责收集2个Agent的输出并整合

结果汇总

- Token消耗：2个Agent的LLM调用 + Manager的协调调用（约3次LLM调用）

与Sequential模式对比

维度	Sequential（程序1）	Hierarchical（程序2-阶段4）
执行方式	任务按顺序执行	任务并行执行
协调者	Crew按Task列表顺序	Manager动态分配
适用场景	有明确先后依赖	任务可独立并行
性能	Task2等Task1完成	2个Task同时进行

小结

通过程序2，我们看到了CrewAI的实战精髓：

- 混合架构：传统代码处理确定性任务，AI处理创造性任务
- Hierarchical模式：Manager协调多个Agent并行工作
- 成本优化：只在必要时启用Crew，节省75%的Token

成本优化

对比两个程序：

- 程序1（Sequential）：全流程AI决策，适合纯创造性任务
- 程序2（混合架构）：按需启用AI，适合复杂业务场景

下一节，我们将回顾整个学习旅程，总结从理论到实践的关键收获。

图片

